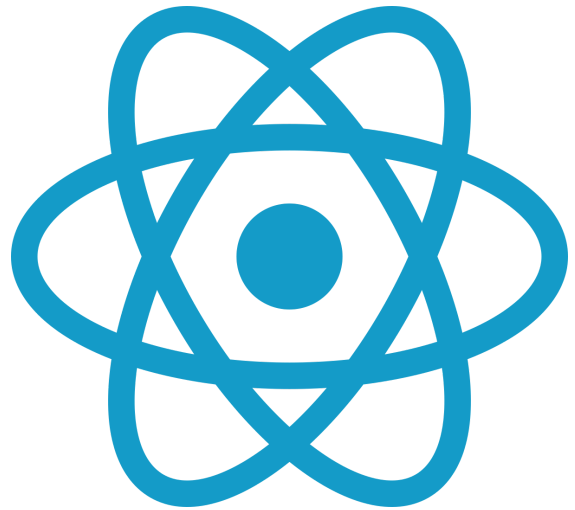


Manual de usuario de la app



Alumna: Nayra Ramírez Jorge
Curso: 2ºA DAM

ÍNDICE

1. Desarrollo de la actividad.....	3
1.1. Creación de la rama crud.....	3
1.2. Conexión del login con la base de datos.....	3
1.3. Creación del menú común a todas las páginas.....	4
1.4. Cómo evitar navegar a /home sin estar logueado: hook useEffect().....	5
1.5. Creación del componente <Dashboard> que usarás en Home.tsx.....	6
1.6. Creación del formulario en Dashboard.tsx.....	7
1.7. Creación del fichero backend/services/items.js.....	8
1.8. Añadir funcionalidad al botón Insertar del formulario.....	8
1.9. Creación del resto de los endpoints: /getItems y /deleteItem.....	9
1.9.1. Obtener datos de la base de datos(/getItems).....	9
1.9.2. Borrar un registro de la base de datos (/deleteItem).....	11
1.10. Creación de la tabla en el fichero Dashboard.tsx: lógica para mostrar los datos en la tabla y lógica para borrar un registro de la tabla.....	12
1.10.1. Lógica para mostrar los datos en la tabla.....	12
1.10.2. Implementación de la tabla: componente Table.....	12
1.10.3. Lógica para borrar un registro de la tabla.....	13
1.11. Creación de ayuda contextual: componente Tooltip.....	14
1.12. Agregar el manual de usuario a tu aplicación.....	15
2. Ejecución del programa.....	17
3. Enlace a GitHub.....	20

1. Desarrollo de la actividad

1.1. Creación de la rama crud

Primero que nada en Visual Studio Code nos situamos en la carpeta llamada proyecto *Tus iniciales* (en mi caso es *proyectornj*) que creamos en actividades anteriores, y escribimos el siguiente comando en la terminal para crear una nueva rama llamada *crud* y, además cambiarnos a esa rama:

```
git checkout -b crud
```

Comprobamos que estamos en esta nueva rama:

```
git branch
```

```
nayramirezjorge@MacBook-Air-de-Nayra ProyectosReact % cd proyectornj
nayramirezjorge@MacBook-Air-de-Nayra proyectornj % git checkout -b crud
Switched to a new branch 'crud'
nayramirezjorge@MacBook-Air-de-Nayra proyectornj % git branch
* crud
  login
  main
  redux
nayramirezjorge@MacBook-Air-de-Nayra proyectornj %
```

1.2. Conexión del login con la base de datos

Con lo explicado en clase sobre el backend y los endpoints, reemplaza tu comprobación de usuario y login correcto con la llamada al endpoint `/login` para conectarte a la base de datos y hacer la autenticación del usuario.

Implementamos la conexión del *login* con la base de datos real, modificando completamente la función *handleSubmit*. Transformamos en una función asíncrona que realiza una petición *HTTP POST* al endpoint `/api/login` del *backend*, enviando las credenciales del usuario en formato *JSON* para su validación en la base de datos *MySQL*.

```
20 const handleSubmit = async (e: any) => {
21   e.preventDefault();
22
23   // Limpiar alertas anteriores
24   setShowSuccess(false);
25   setShowError(false);
26
27   // Mostrar en consola los datos introducidos
28   console.log('Usuario introducido:', username);
29   console.log('Contraseña introducida:', password);
30
31   try {
32     // Llamada al endpoint /login del backend
33     const response = await fetch('/api/login', {
34       method: 'POST',
35       headers: {
36         'Content-Type': 'application/json',
37       },
38       body: JSON.stringify({
39         username: username,
40         password: password
41       })
42     });
43
44     const data = await response.json();
45
46     if (data.success) {
47       console.log('Credenciales CORRECTAS');
48       console.log('Datos del usuario:', data.user);
49       setShowSuccess(true);
50
51       // aquí pongo el dispatch para cambiar el estado a login en el store del redux
52       dispatch(authActions.login({
53         name: data.user.nombre || data.user.username,
54         rol: data.user.rol || 'usuario'
55       }));
56
57       // Navegar a /home después de mostrar el alert brevemente
58       setTimeout(() => {
59         navigate('/home');
60       }, 1000);
61     } else {
62       console.log('Credenciales INCORRECTAS:', data.message);
63       setShowError(true);
64     }
65   } catch (error) {
66     console.error('Error de conexión con el servidor:', error);
67     setShowError(true);
68   }
69 }
```


Modificamos *App.tsx*:

```
proyectorj > Frontend > src > App.tsx > ...
1 import './App.css'
2 import Login from './pages/Login'
3 import Home from './pages/Home'
4 import Reports from './pages/Reports'
5 import ErrorPage from './pages/ErrorPage'
6 import MenuComponent from './components/Menu'
7 import { createBrowserRouter, RouterProvider } from 'react-router-dom'
8 import { Provider } from 'react-redux'
9 import { store } from './store'
10
11 // Layout component que incluye el menú
12 const Layout = ({ children }: { children: React.ReactNode }) => {
13   return (
14     <MenuComponent />
15     {children}
16   )
17 }
18
19
20
21 const router = createBrowserRouter([
22   {
23     path: '/',
24     errorElement: <ErrorPage />,
25     children: [
26       {
27         index: true,
28         element: <Login />
29       },
30       {
31         path: 'home',
32         element: (
33           <Layout>
34             <Home />
35           </Layout>
36         )
37       },
38       {
39         path: 'reports',
40         element: (
41           <Layout>
42             <Reports />
43           </Layout>
44         )
45       }
46     ]
47   },
48 ])
49
50 function App() {
51   return (
52     <Provider store={store}>
53       <RouterProvider router={router} />
54     </Provider>
55   )
56 }
57
58 export default App
```

1.4. Cómo evitar navegar a /home sin estar logueado: hook useEffect()

Para implementar esta protección en el menú, modificamos el *Menu.tsx* añadiendo el hook *useEffect*:

```
proyectorj > Frontend > src > components > Menu.tsx > MenuComponent > handleLogout
1 import React, { useState, useEffect } from 'react'
2 import { Link, useNavigate } from 'react-router-dom'
3 import { useDispatch, useSelector } from 'react-redux'
4 import { RootState } from './store'
5 import { authActions } from './store/authSlice'
6
7 // Componentes de Material-UI
8 import { AppBar, Toolbar, Typography, IconButton, Drawer, List, ListItem, ListItemButton, ListItemIcon, ListItemText, Box, Button,
9   useMediaQuery, useTheme, } from '@mui/material'
10
11 // Icons
12 import MenuIcon from '@mui/icons-material/Menu'
13 import HomeIcon from '@mui/icons-material/Home'
14 import AssessmentIcon from '@mui/icons-material/Assessment'
15 import PersonIcon from '@mui/icons-material/Person'
16
17 const MenuComponent: React.FC = () => {
18   const [drawerOpen, setDrawerOpen] = useState(false)
19
20   const dispatch = useDispatch()
21   const navigate = useNavigate()
22   const theme = useTheme()
23   const isMobile = useMediaQuery(theme.breakpoints.down('md'))
24
25   // Datos del usuario desde Redux
26   const { userName, userRole, isAuthenticated } = useSelector((state: RootState) => state.authenticator)
27
28   useEffect(() => {
29     if (!isAuthenticated) {
30       navigate('/')
31     }
32   }, [isAuthenticated, navigate])
33
34   // Abrir/cerrar drawer móvil
35   const toggleDrawer = (open: boolean) => {
36     setDrawerOpen(open)
37   }
38 }
```

Ahora cuando ponemos <http://localhost:5173/home> o <http://localhost:5173/reports> directamente en la URL, no nos deja entrar puesto que no estamos logueados, y nos redirige a la página de *login*.

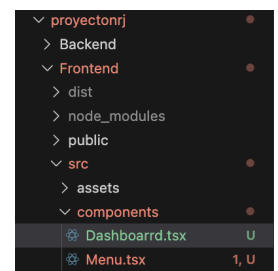
1.5. Creación del componente <Dashboard> que usarás en Home.tsx

En este paso vamos a crear el Dashboard en *Frontend/src/components/Dashboard.tsx*:

Escribimos el código:

Creamos el componente *Dashboard* en el archivo *frontend/src/components/Dashboard.tsx* que contiene la estructura base para el formulario y la tabla que se utilizarán en la página principal. Este componente se renderiza dentro de *Home.tsx* y está compuesto por dos secciones principales: un formulario para agregar datos y una tabla para visualizar la información, ambos envueltos en componentes *Paper* de *Material-UI* para mejorar la presentación visual.

Modificamos *Home.tsx* para importar y utilizar el componente *Dashboard*, eliminando la importación del *Menu* ya que este ahora se maneja a través del layout común en *App.tsx*. La estructura resultante muestra primero la información de bienvenida al usuario (nombre y rol) seguida del dashboard con las herramientas CRUD, manteniendo una organización clara y separada de responsabilidades entre los componentes.



```
proyectorj > Frontend > src > components > Dashboard.tsx > ...
1  import React from 'react';
2  import { Box, Typography, Paper } from '@mui/material';
3
4  const Dashboard: React.FC = () => {
5    return (
6      <Box sx={{ p: 3 }}>
7        <Paper elevation={3} sx={{ p: 3, mb: 3 }}>
8          <Typography variant="h5" component="h2" gutterBottom>
9            Formulario
10          </Typography>
11          <Typography variant="body1">
12            Aquí irá posteriormente el formulario para agregar datos
13          </Typography>
14        </Paper>
15
16        <Paper elevation={3} sx={{ p: 3 }}>
17          <Typography variant="h5" component="h2" gutterBottom>
18            Tabla de Datos
19          </Typography>
20          <Typography variant="body1">
21            Aquí irá posteriormente la tabla para mostrar los datos
22          </Typography>
23        </Paper>
24      </Box>
25    );
26  };
27
28  export default Dashboard;
```

Modificamos *Home.tsx*:

```
proyectorj > Frontend > src > pages > Home.tsx > ...
1  import { useSelector } from 'react-redux';
2  import { RootState } from '../store';
3  import { Typography, Container, Box } from '@mui/material';
4  import Dashboard from '../components/Dashboard';
5
6  export default function Home() {
7    const userData = useSelector((state: RootState) => state.authenticator);
8
9    return (
10      <Container maxWidth="lg">
11        <Box sx={{ my: 4 }}>
12          <Typography
13            variant="h2"
14            component="h1"
15            gutterBottom
16            color="primary"
17            sx={{ textAlign: 'center', mt: 4 }}
18          >
19            Página Home de Nayra
20          </Typography>
21
22          <Typography
23            variant="h4"
24            component="h2"
25            sx={{ textAlign: 'center', mt: 4, color: 'secondary.main' }}
26          >
27            Bienvenido, {userData.userName}
28          </Typography>
29
30          <Typography
31            variant="h5"
32            component="h3"
33            sx={{ textAlign: 'center', mt: 2 }}
34          >
35            Tu rol es: {userData.userRol}
36          </Typography>
37
38          <Dashboard />
39        </Box>
40      </Container>
41    );
42  }
```

1.6. Creación del formulario en Dashboard.tsx

Modificamos *Dashboard.tsx*:

Implementamos un formulario completo en el componente *Dashboard* que permite insertar nuevos registros en la base de datos. El formulario incluye campos para nombre, marca, tipo y precio, utilizando una *interface TypeScript* para definir la estructura de los datos y estados *React* para gestionar los valores del formulario y las alertas de *feedback*. Al enviar el formulario, se realiza una petición *POST* al *endpoint* del *backend* para guardar los datos, mostrando mensajes de éxito o error según el resultado de la operación, y se limpian automáticamente los campos tras una inserción exitosa.

```

import React, { useState } from 'react';
import {
  Box,
  Paper,
  TextField,
  Button,
  Grid,
  Alert
} from '@mui/material';

interface ItemType {
  id: number;
  nombre: string;
  marca: string;
  tipo: string;
  precio: number;
}

const itemInitialState: ItemType = {
  nombre: '',
  marca: '',
  tipo: '',
  precio: 0
};

const Dashboard: React.FC = () => {
  const [item, setItem] = useState<ItemType>(itemInitialState);
  const [showSuccess, setShowSuccess] = useState(false);
  const [showError, setShowError] = useState(false);

  const handleInputChange = (e: React.ChangeEvent) => {
    const { name, value } = e.target;
    setItem(prevItem => ({
      ...prevItem,
      [name]: name === 'precio' ? parseFloat(value) || 0 : value
    }));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();

    setShowSuccess(false);
    setShowError(false);

    try {
      const response = await fetch('http://localhost:5173/items', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify(item)
      });

      if (response.ok) {
        setShowSuccess(true);
        setItem(itemInitialState);
      } else {
        setShowError(true);
      }
    } catch (error) {
      console.error('Error al insertar datos:', error);
      setShowError(true);
    }
  };

  return (
    <Box sx={{ p: 3 }}>
      {/*Formulario completo para insertar datos */}
      <Paper elevation={3} sx={{ p: 3, mt: 3 }}>
        {/*Formulario variante "h1" componente "h2" gutterButton*/}
        Formulario para Insertar Datos
        </Typography>

        {/*Formulario con Grid responsive */}
        <form onSubmit={handleSubmit}>
          <Grid container spacing={2}>
            <Grid item xs={12} sm={6}>
              <TextField
                fullWidth
                label="Nombre"
                name="nombre"
                value={item.nombre}
                onChange={handleInputChange}
                required
              />
            </Grid>
            <Grid item xs={12} sm={6}>
              <TextField
                fullWidth
                label="Marca"
                name="marca"
                value={item.marca}
                onChange={handleInputChange}
                required
              />
            </Grid>
            <Grid item xs={12} sm={6}>
              <TextField
                fullWidth
                label="Tipo"
                name="tipo"
                value={item.tipo}
                onChange={handleInputChange}
                required
              />
            </Grid>
            <Grid item xs={12} sm={6}>
              <TextField
                fullWidth
                label="Precio"
                name="precio"
                type="number"
                value={item.precio}
                onChange={handleInputChange}
                inputProps={{ step: "0.01" }}
                required
              />
            </Grid>
            <Grid item xs={12}>
              <Button
                type="submit"
                variant="contained"
                color="primary"
                size="large">
                  + INSERTAR DATOS
                </Button>
              </Grid>
            </form>

            {/* *** NUEVO: Alertas de éxito y error */}
            {showSuccess &&
              <Alert severity="success" sx={{ mt: 2 }}>
                Datos insertados correctamente en la base de datos
              </Alert>
            }
            {showError &&
              <Alert severity="error" sx={{ mt: 2 }}>
                Error al insertar los datos
              </Alert>
            }
          </Paper>

          {/* Parte existente - Tabla de Datos */}
          <Table>
            <Caption>Tabla de Datos</Caption>
            <TableBody>
              <TableRow>
                <th>ID</th>
                <th>Nombre</th>
                <th>Marca</th>
                <th>Tipo</th>
                <th>Precio</th>
              </TableRow>
              <{ /* Irá posteriormente la tabla para mostrar los datos */ >
            </TableBody>
          </Table>
        </Box>
      </div>
    );
  };

export default Dashboard;

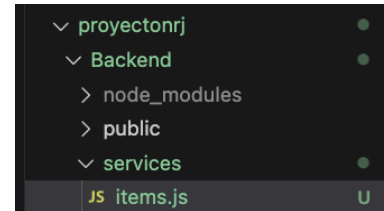
```

1.7. Creación del fichero backend/services/items.js

En este paso vamos a crear *Backend/services/items.js*:

Escribimos el código:

```
projectonrj > Backend > services > JS items.js > [e] <unknown>
1  const db = require('../db')
2  const helper = require('../helper')
3  const config = require('../config')
4
5  // Función para insertar datos
6  async function insertData(req, res) {
7    const data = req.body
8    const result = await db.query(
9      'INSERT INTO coleccion (nombre, marca, tipo, precio)
10     VALUES (?, ?, ?, ?)',
11     [data.nombre, data.marca, data.tipo, data.precio]
12   )
13   return result.affectedRows
14 }
15
16 // Función para obtener datos
17 async function getData(req, res) {
18   const rows = await db.query('SELECT * FROM coleccion')
19   const data = helper.emptyOrRows(rows)
20   return { data }
21 }
22
23 // Función para borrar datos
24 async function deleteData(req, res) {
25   const data = req.query
26   const result = await db.query(
27     'DELETE FROM coleccion WHERE id = ?',
28     [data.id]
29   )
30   return result.affectedRows
31 }
32
33 module.exports = {
34   getData,
35   insertData,
36   deleteData
37 }
```



En la terminal del dispositivo comprobamos que se muestra la información del archivo *bdgestion.sql*:

```
nayramirezjorge@MacBook-Air-de-Nayra projectonrj % mysql -u root bdgestion < bdgestion.sql
nayramirezjorge@MacBook-Air-de-Nayra projectonrj % mysql -u root -e "USE bdgestion; SHOW TABLES;"
+-----+
| Tables_in_bdgestion |
+-----+
| coleccion             |
| usuarios              |
+-----+
nayramirezjorge@MacBook-Air-de-Nayra projectonrj % mysql -u root -e "USE bdgestion; SELECT * FROM usuarios;"
+----+-----+-----+-----+-----+
| id | nombre | login | password | rol |
+----+-----+-----+-----+-----+
| 1  | Patricia | patricia | 123456789 | admin |
| 2  | usuario | user | 123456789 | user |
+----+-----+-----+-----+-----+
nayramirezjorge@MacBook-Air-de-Nayra projectonrj %
```

1.8. Añadir funcionalidad al botón Insertar del formulario

Modificamos el archivo que acabamos de crear:

```
projectonrj > Backend > services > JS items.js > [e] deleteData
1  const db = require('../db')
2  const helper = require('../helper')
3  const config = require('../config')
4
5  // Función para insertar datos
6  async function insertData(req, res) {
7    // data tiene los datos que vamos a insertar en la base de datos
8    const data = req.query
9    const result = await db.query(
10     'INSERT INTO coleccion (nombre, marca, tipo, precio) VALUES (?, ?, ?, ?)',
11     [data.nombre, data.marca, data.tipo, data.precio]
12   )
13   // Si affectedRows > 0 significa que se insertó correctamente
14   return result.affectedRows
15 }
16
17 // Función para obtener datos
18 async function getData(req, res) {
19   const rows = await db.query('SELECT * FROM coleccion')
20   const data = helper.emptyOrRows(rows)
21   return { data }
22 }
23
24 // Función para borrar datos
25 async function deleteData(req, res) {
26   const data = req.query
27   const result = await db.query(
28     'DELETE FROM coleccion WHERE id = ?',
29     [data.id]
30   )
31   return result.affectedRows
32 }
33
34 module.exports = {
35   getData,
36   insertData,
37   deleteData
38 }
```

Ahora añadimos el archivo `index.js` proporcionado por la profesora en *Backend*, y le añadimos lo siguiente:

```
proyectorj > Backend > JS index.js > ...
1 //Importo el express y el cors
2 const express = require('express')
3 const cors = require('cors')
4 //Importo el fichero login.js que está en la carpeta services
5 const login = require('./services/login')
6 //Importar items
7 const items = require('./services/items')
8
9 //Definimos el puerto por que va a escuchar nuestra API las peticiones
10 const port = 3030
11
12 const app = express()
13 app.use(express.json())
14 app.use(
15   express.urlencoded({
16     extended: true
17   })
18 )
19 app.use(cors())
20
21 //Ejemplo para ver cómo funciona un endpoint:
22 //Este endpoint / y devuelve un mensaje
23 app.get('/', function (req, res) {
24   res.json({message: 'Hola Mundo!'})
25 })
26
27 //Creación del endpoint: /login
28 //Llama al fichero login.js usando el método getUserData pasándole
29 //el login (user) y la contraseña (password)
30 app.get('/login', async function(req, res, next) {
31   console.log(req.query)
32   console.log(req.query.user)
33   console.log(req.query.password)
34   try {
35     res.json(await login.getUserData(req.query.user, req.query.password))
36   } catch (err) {
37     console.error('Error while getting data ', err.message);
38     next(err);
39   }
40 })
41
42 //Endpoint para insertar items
43 app.get('/addItem', async function(req, res, next) {
44   console.log(req.query)
45   try {
46     res.json(await items.insertData(req, res))
47   } catch (err) {
48     console.error('Error while inserting items', err.message);
49     next(err);
50   }
51 })
52
53 //Iniciamos la API
54 app.listen(port)
55 console.log('API escuchando en el puerto ' + port)
```

Cambiamos en el archivo `Dashboard.tsx` para que escuche el puerto 3030 en vez de por el 5173 como teníamos antes:

```
46   try {
47     const response = await fetch('http://localhost:3030/items', {
48       method: 'POST',
49       headers: {
50         'Content-Type': 'application/json',
51       },
52       body: JSON.stringify(item)
53     });
```

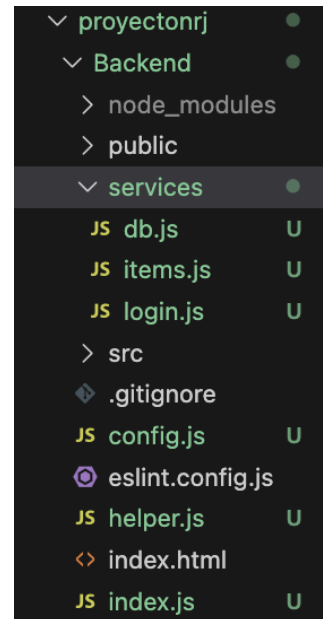
1.9. Creación del resto de los endpoints: /getItem y /deleteItem

1.9.1. Obtener datos de la base de datos(/getItem)

Modificamos el `index.js`, añadiendo `/getItem` y `/deleteItem`:

```
proyectorj > Backend > JS index.js > ...
30 app.get('/login', async function(req, res, next) {
31   console.error('Error while getting data ', err.message);
32   next(err);
33 }
34 })
35
36 //Endpoint para insertar items
37 app.get('/addItem', async function(req, res, next) {
38   console.log(req.query)
39   try {
40     res.json(await items.insertData(req, res))
41   } catch (err) {
42     console.error('Error while inserting items', err.message);
43     next(err);
44   }
45 })
46
47 //Endpoint para obtener todos los items
48 app.get('/getItem', async function(req, res, next) {
49   try {
50     res.json(await items.getData(req, res))
51   } catch (err) {
52     console.error('Error while getting items', err.message);
53     next(err);
54   }
55 })
56
57 //Endpoint para eliminar items
58 app.get('/deleteItem', async function(req, res, next) {
59   console.log(req.query)
60   try {
61     res.json(await items.deleteData(req, res))
62   } catch (err) {
63     console.error('Error while deleting items', err.message);
64     next(err);
65   }
66 })
67
68 //Iniciamos la API
69 app.listen(port)
70 console.log('API escuchando en el puerto ' + port)
```

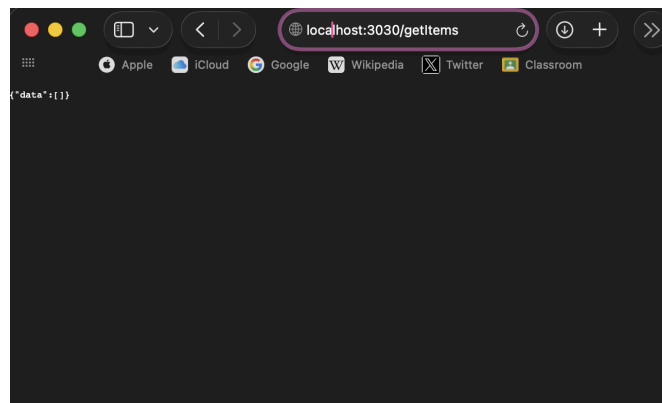
Añadimos los archivos proporcionados por la profesora:



Para comprobar que funciona debemos de ir a la Terminal del dispositivo y poner los siguientes comandos:

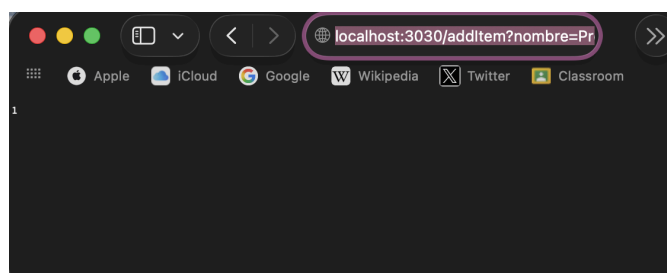
```
Last login: Tue Nov 25 10:22:34 on ttys010
nayraramirezjorge@MacBook-Air-de-Nayra ~ % cd /Users/nayraramirezjorge/Desktop/P
royectosReact/proyectornj/Backend
nayraramirezjorge@MacBook-Air-de-Nayra Backend % node index.js
API escuchando en el puerto 3030
```

Inmediatamente vamos al navegador y ponemos lo siguiente:

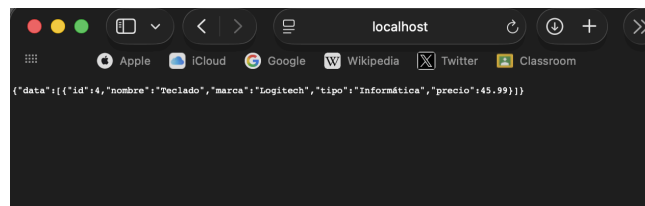


Para añadir un registro a la tabla ponemos, por ejemplo, lo siguiente en el navegador:
<http://localhost:3030/addItem?nombre=Teclado&marca=Logitech&tipo=Informática&precio=45.99>

Le damos al enter y nos saldrá un 1:



Ahora si volvemos a ejecutar <http://localhost:3030/getItems> nos saldrá algo como lo siguiente:



```
{
  "data": [
    {
      "id": 4,
      "nombre": "Teclado",
      "marca": "Logitech",
      "tipo": "Informática",
      "precio": 45.99
    }
  ]
}
```

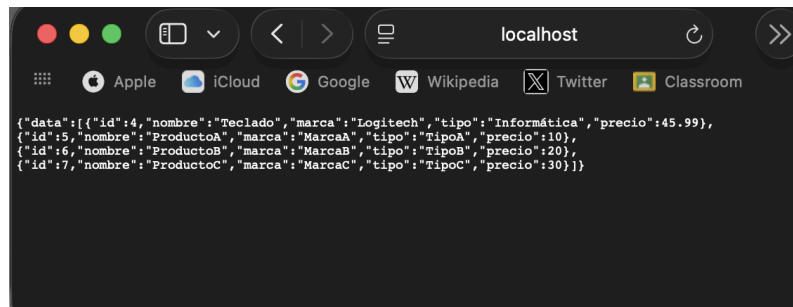
1.9.2. Borrar un registro de la base de datos (/deleteItem)

Para probar que nos funciona el borrar items, vamos a insertar varios, como en el apartado anterior:

<http://localhost:3030/addItem?nombre=ProductoA&marca=MarcaA&tipo=TipoA&precio=10>

<http://localhost:3030/addItem?nombre=ProductoB&marca=MarcaB&tipo=TipoB&precio=20>

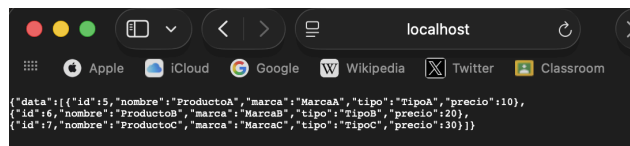
<http://localhost:3030/addItem?nombre=ProductoC&marca=MarcaC&tipo=TipoC&precio=30>



```
{
  "data": [
    {
      "id": 4,
      "nombre": "Teclado",
      "marca": "Logitech",
      "tipo": "Informática",
      "precio": 45.99
    },
    {
      "id": 5,
      "nombre": "ProductoA",
      "marca": "MarcaA",
      "tipo": "TipoA",
      "precio": 10
    },
    {
      "id": 6,
      "nombre": "ProductoB",
      "marca": "MarcaB",
      "tipo": "TipoB",
      "precio": 20
    },
    {
      "id": 7,
      "nombre": "ProductoC",
      "marca": "MarcaC",
      "tipo": "TipoC",
      "precio": 30
    }
  ]
}
```

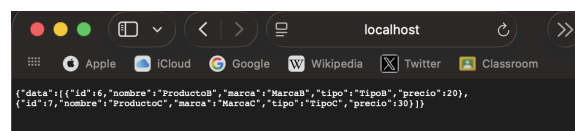
Vamos a probar borrando el id 4 y el 5:

<http://localhost:3030/deleteItem?id=4>



```
{
  "data": [
    {
      "id": 5,
      "nombre": "ProductoA",
      "marca": "MarcaA",
      "tipo": "TipoA",
      "precio": 10
    },
    {
      "id": 6,
      "nombre": "ProductoB",
      "marca": "MarcaB",
      "tipo": "TipoB",
      "precio": 20
    },
    {
      "id": 7,
      "nombre": "ProductoC",
      "marca": "MarcaC",
      "tipo": "TipoC",
      "precio": 30
    }
  ]
}
```

<http://localhost:3030/deleteItem?id=5>



```
{
  "data": [
    {
      "id": 6,
      "nombre": "ProductoB",
      "marca": "MarcaB",
      "tipo": "TipoB",
      "precio": 20
    },
    {
      "id": 7,
      "nombre": "ProductoC",
      "marca": "MarcaC",
      "tipo": "TipoC",
      "precio": 30
    }
  ]
}
```


1.10.3. Lógica para borrar un registro de la tabla

En el código del `TableBody` insertamos un componente `<Button>` con un componente icono (`<DeleteForeverIcon />`). Al picar en dicho botón icono, implementamos una función a la que pasamos el id del registro que queremos borrar. Dicha función hace el *fetch* al *endpoint* `/deleteItem`.

Importamos lo siguiente:

```
proyectorj > Frontend > src > components > Dashboard.tsx > ~ ItemType
1 import React, { useState, useEffect } from 'react';
2 import DeleteForeverIcon from '@mui/icons-material/DeleteForever';
3 import {
4   Box,
```

Y añadimos todo lo siguiente:

```
proyectorj > Frontend > src > components > Dashboard.tsx > ~ Dashboard
35 const Dashboard: React.FC = () => {
36   const [tableData, setTableData] = useState<ItemType[]>([]);
37   const [showError, setShowError] = useState<boolean>(false);
38   const handleDeleteItem = async (row: ItemType) => {
39     if (!row.id) return;
40     try {
41       const response = await fetch('http://localhost:3030/deleteItem?id=${row.id}');
42       const result = await response.json();
43       if (result > 0) {
44         alert('Registro eliminado correctamente');
45         fetchTableData();
46       } else {
47         alert('Error al eliminar el registro');
48       }
49     } catch (error) {
50       console.error('Error al eliminar:', error);
51       alert('Error al eliminar el registro');
52     }
53   };
54   return (
55     <Box sx={{ p: 3 }}>
56       <Paper elevation={3} sx={{ p: 3, mb: 3 }}>
57         <Typography variant="h2" component="h2" gutterBottom>
58           Formulario para Insertar Datos
59         </Typography>
60         <form onSubmit={handleSubmit}>
61           <Grid container spacing={2}>
62             <Grid item xs={12} sm={6}>
63               <TextField
64                 fullWidth
65                 label="Nombre"
66                 name="nombre"
67                 value={item.nombre}
68                 onChange={handleInputChange}
69                 required
70               />
71             </Grid>
72             <Grid item xs={12} sm={6}>
73               <TextField
74                 fullWidth
75                 label="Marca"
76                 name="marca"
77                 value={item.marca}
78                 onChange={handleInputChange}
79                 required
80               />
81             </Grid>
82             <Grid item xs={12} sm={6}>
83               <TextField
```

```
205 <TableBody>
206   {tableData.map((row: ItemType) => (
207     <TableRow key={row.id}>
208       <TableCell>{row.id}</TableCell>
209       <TableCell>{row.nombre}</TableCell>
210       <TableCell>{row.marca}</TableCell>
211       <TableCell>{row.tipo}</TableCell>
212       <TableCell>{row.precio}</TableCell>
213       <TableCell>
214         <Button
215           color="error"
216           onClick={() => handleDeleteItem(row)}
217           startIcon={<DeleteForeverIcon />}
218         >
219           Eliminar
220         </Button>
221       </TableCell>
222     </TableRow>
223   ))}
224 </TableBody>
225 </Table>
226 </TableContainer>
227
228 {tableData.length === 0 && (
229   <Typography variants="body1" sx={{ textAlign: 'center', mt: 2 }}>
230     No hay datos para mostrar
231   </Typography>
232 )}
233 </Paper>
234 </Box>
235 );
236
237 export default Dashboard;
```

1.11. Creación de ayuda contextual: componente Tooltip

Debes seguir trabajando en la rama **informes** del gitHub.

Usando el componente **Tooltip** de la librería **@MUI** crea ayuda contextual en todos los botones de tu aplicación, tanto los que tienen icono como los que no.

Debes indicar la posición del **Tooltip** y hacerlos en forma de flecha. Para ello, lee la documentación siguiente: <https://mui.com/material-ui/react-tooltip/>

Comenzamos importando *Tooltip* en *Dashboard.tsx*:

```
Frontend > src > components > Dashboard.tsx > ItemType > id
1  import React, { useState, useEffect } from 'react';
2  import {
3    Box,
4    Typography,
5    Paper,
6    TextField,
7    Button,
8    Grid,
9    Alert,
10   Table,
11   TableBody,
12   TableCell,
13   TableContainer,
14   TableHead,
15   TableRow,
16   Tooltip,
17 } from '@mui/material';
```

Vamos añadiendo los *Tooltip*, en la siguiente imagen se le añade al botón + *INSERTAR DATOS*:

```
165 <Grid item xs={12}>
166   {/*ponemos arrow para que tenga una flecha y placement para colocar el tooltip en la parte superior*/}
167   <Tooltip title="Se añadirán los datos a la base de datos" arrow placement="top">
168     <Button
169       type="submit"
170       variant="contained"
171       color="primary"
172       size="large"
173     >
174       + INSERTAR DATOS
175     </Button>
176   </Tooltip>
177 </Grid>
```

Los añadimos a todos los botones:

```
222 {/* Botón con DeleteForeverIcon y texto Eliminar */}
223 <Tooltip title="Se eliminarán los datos definitivamente" arrow placement="right">
224   <Button
225     color="error"
226     onClick={() => handleDeleteItem(row)}
227     startIcon={DeleteForeverIcon} />
228   >
229     Eliminar
230   </Button>
231 </Tooltip>
```

Ahora hacemos lo mismo pero con los botones del menú hamburguesa que se encuentra en el lado superior izquierdo.

Importamos *Tooltip*:

```
Frontend > src > components > Menu.tsx > ...
1  import React, { useState, useEffect } from 'react';
2  import { Link, useNavigate } from 'react-router-dom';
3  import { useDispatch, useSelector } from 'react-redux';
4  import { RootState } from '../store';
5  import { authActions } from '../store/authSlice';
6
7  import { AppBar, Toolbar, Typography, IconButton, Drawer, List, ListItem, ListItemButton, ListItemIcon, ListItemText,
8    Box, useMediaQuery, useTheme, Divider, Tooltip } from '@mui/material';
```

Como antes, implementamos *Tooltip* a todos los botones:

- Menú hamburguesa :

```
123      {/* Botón hamburguesa */}
124      <Tooltip title="Abrir menú de navegación" arrow placement="bottom-end">
125        <IconButton
126          size="large"
127          edge="start"
128          color="inherit"
129          aria-label="menu"
130          sx={{ mr: 2 }}
131          onClick={toggleDrawer(true)}
132        >
133          <MenuIcon />
134        </IconButton>
135      </Tooltip>
```

- Inicio:

```
80      <Tooltip title="Ir a la página de inicio" arrow placement="bottom">
81        <Link to="/home" style={{ textDecoration: 'none', color: 'inherit', width: '100%' }}>
82          <ListItemButton>
83            <ListItemIcon>
84              <HomeIcon />
85            </ListItemIcon>
86            <ListItemText primary="Inicio" />
87          </ListItemButton>
88        </Link>
89      </Tooltip>
```

- Informes:

```
94      <Tooltip title="Ir a la página de informes" arrow placement="bottom">
95        <Link to="/reports" style={{ textDecoration: 'none', color: 'inherit', width: '100%' }}>
96          <ListItemButton>
97            <ListItemIcon>
98              <AssessmentIcon />
99            </ListItemIcon>
100            <ListItemText primary="Informes" />
101          </ListItemButton>
102        </Link>
103      </Tooltip>
```

- Cerrar sesión:

```
112      <Tooltip title="Cerrar sesión" arrow placement="bottom">
113        <ListItemButton onClick={handleLogout}>
114          <ListItemIcon>
115            <ExitToAppIcon />
116          </ListItemIcon>
117          <ListItemText primary="Cerrar Sesión" />
118        </ListItemButton>
119      </Tooltip>
```

1.12. Agregar el manual de usuario a tu aplicación

Has estado haciendo un manual de usuario para tu aplicación. Ahora es el momento de añadirlo a la misma. Lo vas a hacer en el link de **Ayuda**

Queremos que cuando el usuario de la aplicación pique en **Ayuda** se abra una página nueva a parte de la aplicación donde se muestre el PDF del manual de ayuda. Para poder hacer eso tenemos que:

```
<Link to={`/${NombreDocumentoPDF}`} target='_blank'>Ayuda</Link>
```

El **target** tiene que ser `_blank`. De esta forma le dices que se abra una página nueva a parte.

El **to** tiene que ser al manual de ayuda PDF que has generado y que debes añadir a la carpeta frontend/public de forma que la aplicación lo encuentre ahí.

Como lo que se va a abrir al picar en **Ayuda** es un documento PDF, **no** hace falta añadir el enrutamiento en frontend/App.tsx

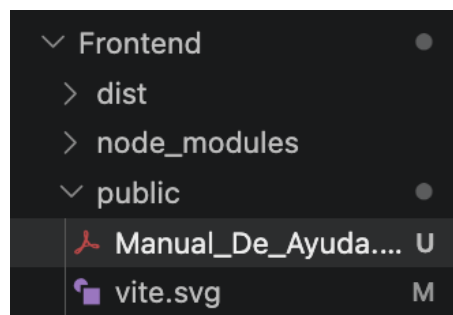
Para hacer todo lo solicitado comenzamos creando un botón en el menú lateral llamado “Ayuda”, le ponemos el *Tooltip* y el enlace al pdf de manual de ayuda (el de la UT2A4).

```
107 <ListItem disablePadding>
108   <Tooltip title="Abrir el manual de ayuda" arrow placement="bottom">
109     <Link to="/Ramirez_Jorge_Nayra_UT2A4.pdf" target="_blank" style={{ textDecoration: 'none', color: 'inherit', width: '100%' }}>
110       <ListItemButton>
111         <ListItemIcon>
112           <HelpIcon />
113         </ListItemIcon>
114         <ListItemText primary="Ayuda" />
115       </ListItemButton>
116     </Link>
117   </Tooltip>
118 </ListItem>
```

Añadimos el import de *HelpIcon* al inicio de la clase para que se vea el icono de un signo de interrogación (?) al lado de “Ayuda”:

```
12 import MenuIcon from '@mui/icons-material/Menu';
13 import HomeIcon from '@mui/icons-material/Home';
14 import AssessmentIcon from '@mui/icons-material/Assessment';
15 import PersonIcon from '@mui/icons-material/Person';
16 import ExitToAppIcon from '@mui/icons-material/ExitToApp';
17 import HelpIcon from '@mui/icons-material/Help';
```

Incorporamos el pdf del manual de ayuda en *Frontend/public/Manual_De_Ayuda.pdf*:



2. Ejecución del programa

Una vez iniciamos sesión, entramos en la página principal donde podemos gestionar la base de datos.

Página Home de Nayra

Bienvenido, nayra
Tu rol es: administrador

Formulario para Insertar Datos

Nombre * Marca *

Tipo * Precio *

+ INSERTAR DATOS

Tabla de Datos

ID	Nombre	Marca	Tipo	Precio	Acciones
No hay datos para mostrar					

Al pulsar en + INSERTAR DATOS:

Página Home de Nayra

Bienvenido, nayra
Tu rol es: administrador

Formulario para Insertar Datos

Nombre * Manzana Marca * Marlene

Tipo * Fruta Precio * 1.04

+ INSERTAR DATOS

Datos guardados con éxito

Cerrar

Tabla de Datos

ID	Nombre	Marca	Tipo	Precio	Acciones
----	--------	-------	------	--------	----------

Al insertar datos

Página Home de Nayra

Bienvenido, nayra
Tu rol es: administrador

Formulario para Insertar Datos

Nombre * Marca *

Tipo * Precio *

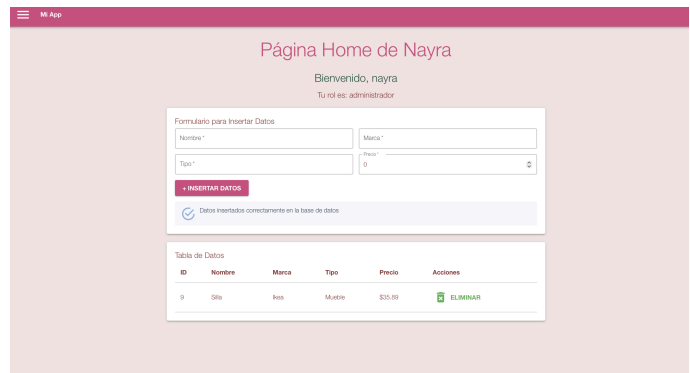
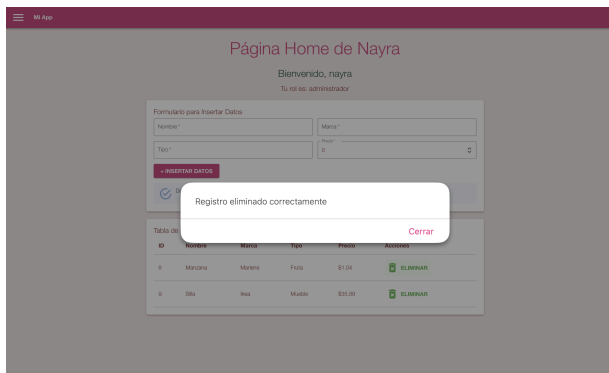
+ INSERTAR DATOS

Datos insertados correctamente en la base de datos

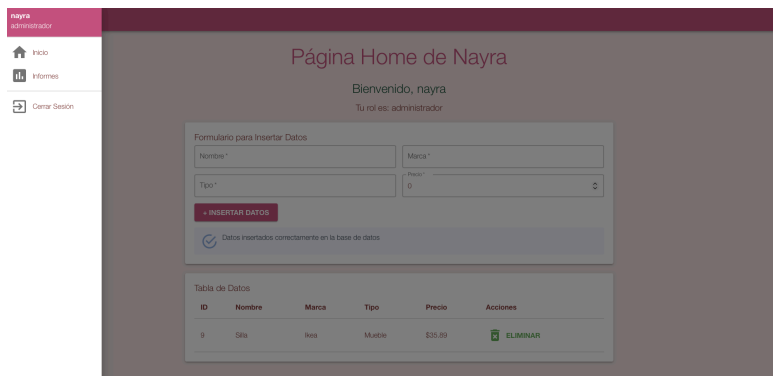
Tabla de Datos

ID	Nombre	Marca	Tipo	Precio	Acciones
8	Manzana	Marlene	Fruta	\$1.04	ELIMINAR
9	Sila	Ikea	Mueble	\$35.99	ELIMINAR

Al pulsar en **ELIMINAR**:

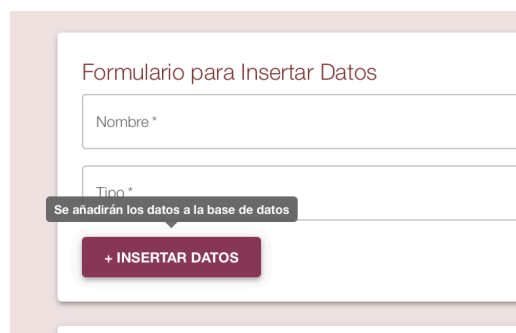


Menú lateral (hamburguesa)

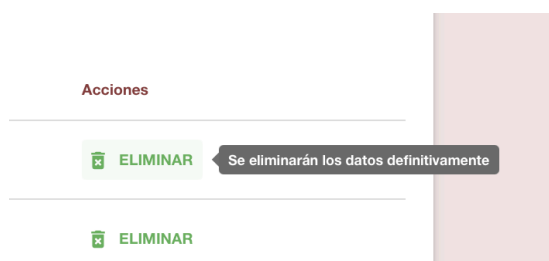


Al haber implementado el Tooltip, cada vez que pongamos el ratón encima de un botón, se mostrará el este en forma de flecha, como hemos implementado en el código.

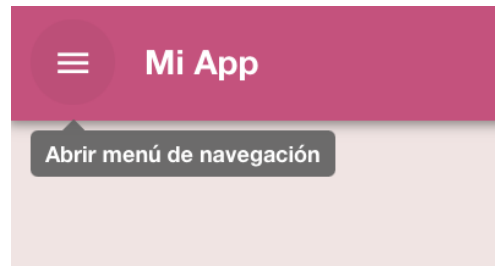
- + Insertar datos:



- Eliminar



- Menú hamburguesa:



- Inicio:



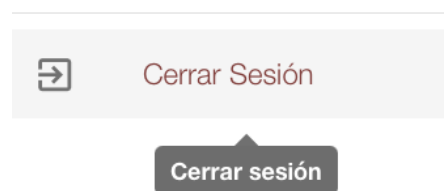
- Informes:



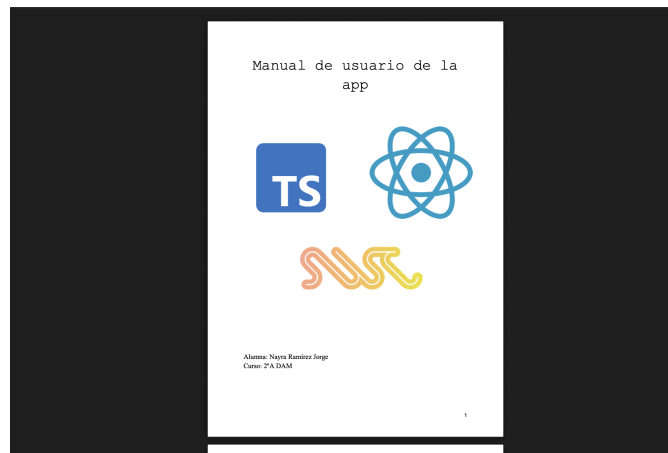
- Ayuda:



- Cerrar sesión:



Si pulsamos en el botón “Ayuda” se nos abrirá el manual de ayuda para el usuario:



3. Enlace a GitHub

<https://github.com/NRJ0209/proyectornrj.git>